Republic of the Philippines
**JOSE RIZAL MEMORIAL STATE UNIVERSITY**
The Premiere University in Zamboanga del Norte
**COLLEGE OF ENGINEERING**

# COMPREHENSIVE ASSESSMENT AND PREPARATION SYSTEM

## (CAPS-Mobile App)

## Docker Errors Resolution Report

Group: Ace and Queens

| Docker Errors / Issues | Why the Error Occurred? | Complete Steps for Solving the Error |
|---|---|---|
| **1. APP_KEY is missing — backend refuses to start** | The backend now intentionally refuses to start if APP_KEY is absent from the .env file. This protects encrypted database data — silently generating a new key could permanently break access to existing encrypted records. | 1. Confirm the running container name:<br>docker compose -f docker-compose.yml ps<br>2. Read startup logs to confirm the error:<br>docker logs caps_backend_mobile<br>3. Open Backend - Deployment/.env and restore the original APP_KEY that matches the database.<br>4. For a brand-new database only, generate a fresh key:<br>docker run --rm --entrypoint php caps-mobile-backend artisan key:generate --show<br>5. Restart the backend:<br>docker compose -f docker-compose.yml up -d backend<br>6. Verify the API responds:<br>curl http://localhost:8005/api/app-version |
| **2. Wrong container name used in commands (caps_backend vs** | Older project documents still reference caps_backend, but the current docker-compose.yml now uses caps_backend_mobile. Commands | 1. Always confirm the actual container name before running any command:<br>docker compose -f docker-compose.yml ps<br>2. Use caps_backend_mobile for |

| | | |
|---|---|---|
| **caps_backend_mobile)** | targeting the old name will fail or hit the wrong container. | all commands going forward.<br>3. Update any team scripts or older docs that still hardcode caps_backend. |
| **3. Encrypted data broken after container rebuild** | When a new APP_KEY is generated against a MariaDB volume that already has encrypted rows, Laravel can no longer decrypt that data. This happens when someone rebuilds the container and creates a fresh key without restoring the original. | 1. Never generate a new APP_KEY if reusing an existing database volume.<br>2. Treat APP_KEY and the database volume as a matched pair — they must always go together.<br>3. Keep a secure backup of Backend - Deployment/.env at all times.<br>4. To recover, restore the original key into .env then restart:<br>docker compose -f docker-compose.yml up -d backend |
| **4. .env file missing or not loaded by Docker** | The backend image intentionally excludes .env via .dockerignore — it is never baked into the image. Docker reads it at runtime through env_file in docker-compose.yml. If the file is missing or empty, the backend won't get the values it needs. | 1. Do not copy .env into the Docker image — this is intentional for security.<br>2. Make sure Backend - Deployment/.env exists on the host with all values filled in.<br>3. Note: .env.example is only a template — copying it alone is not enough because APP_KEY will still be empty.<br>4. After creating or updating .env, restart the backend:<br>docker compose -f |

| | | |
|---|---|---|
| | | docker-compose.yml up -d backend |
| **5. Login Failed: Invalid Credentials (After Migration)** | You have the database tables (migration was done), but they are completely empty — there are no user accounts yet. No Admin or Test User exists to log in with.<br><br>The deployment manual tells you to run a db:seed command that fills in starter accounts after migration. If that step was skipped, no one can log in. | 1. Open Command Prompt as Administrator.<br>2. Enter the backend container:<br>**docker exec -it caps_backend bash**<br>3. Run the seeder command to add the default users into the database:<br>**php artisan db:seed**<br>4. Wait until you see: "Database seeding completed successfully."<br>5. If you get a duplicate entry error (data already exists), run this instead to reset and reseed:<br>**php artisan migrate:fresh --seed**<br>6. Type exit to leave the container:<br>**exit**<br>7. Go back to the CAPS login page and try logging in again with the default admin account. |
| **6. Vite manifest not found (Frontend Error)** | The React code is written, but it hasn't been "built" into a format that the web browser can understand. | 1. In your frontend terminal, run npm run dev (for live editing).<br>2. Or run npm run build to create the final static files. |
| **7. 403 Forbidden or Permission Denied** | Docker is trying to write logs or save files to your laptop's storage, but your computer's security is blocking it. | 1. Run chmod -R 775 storage bootstrap/cache (Linux/Mac).<br>2. On Windows, ensure your |

| | | |
|---|---|---|
| | | project folder is not "Read Only" in the file properties. |
| **8. Missing Dependencies (404/500)** | The project is a "skeleton" without its library files. | 1. Run composer install (Backend).<br>2. Run npm install (Frontend). |
| **9. Encryption Key Missing** | Laravel requires a unique APP_KEY to encrypt user sessions/passwords. | 1. Run php artisan key:generate.<br>2. This adds the key to your .env file automatically. |
| **10. Target Class/Table Not Found** | The Database is "Up" but empty; the "filing cabinet" has no drawers. | 1. Run php artisan migrate.<br>2. This builds the tables (users, assessments, etc.) in MySQL. |
| **11. Login "Invalid Credentials"** | The system has tables but no data (no Admin user to log in with). | 1. Run php artisan db:seed.<br>2. This creates the default "baseline" users for testing. |
| **12. Vite Manifest Missing** | The React frontend code is written but hasn't been compiled for the browser. | 1. Run npm run dev for development.<br>2. Use npm run build for the final production version. |
| **13. CORS / API Connection Refused** | The Frontend and Backend are on different "islands" and aren't allowed to talk. | 1. Update config/cors.php to allow the frontend URL.<br>2. Ensure SANCTUM_STATEFUL_DOMAINS in .env includes your local IP/URL. |
| **14. Permission Denied (storage/logs)** | The Docker container doesn't have "write access" to your computer's folders. | 1. Run chmod -R 775 storage bootstrap/cache.<br>2. Ensure the user running Docker has owner permissions. |
| **15. Parse error: syntax error** | The Windows/Local terminal tried to read symbols (like $) instead of sending them to Docker. | 1. Wrap the command in single quotes: php -r 'code here' |

| | | |
|---|---|---|
| **16. apache2 restart: failed!** | The web server crashed or locked up while trying to update its internal configuration. | 1. docker stop caps-backend<br>2. docker start caps-backend |
| **17. CORS error / Preflight** | The browser blocked Port 5173 from talking to Port 8000 because they lacked a "trust" handshake. | 1. docker exec -it caps-backend sh -c "echo 'CORS_ALLOWED_ORIGINS=http://localhost:5173' >> .env"<br>2. php artisan config:clear |
| **18. Something went wrong!** | General failure often caused by missing folder permissions or a missing security key. | 1. php artisan key:generate<br>2. chmod -R 777 storage bootstrap/cache<br>3. **Hard Reload** (F12 > Right-click Refresh). |
| **19. CAPS-Backend Container Keeps Stopping** | When you run the compose stack, MySQL (the database) takes longer to fully start than the backend. The backend tries to connect to the database right away, but since the database is not ready yet, the backend fails and shuts itself down. | 1. Run the compose stack first by opening Command Prompt as Administrator and typing:<br>**docker-compose up -d --build**<br>2. Wait about 30 seconds to give MySQL enough time to fully start up.<br>3.After waiting, go to Docker Desktop and check if caps_backend is still stopped (red).<br>4. If it is stopped, manually start it by typing:<br>**docker start caps_backend**<br>5. Check Docker Desktop again — the backend should now show as Running (green).<br>6. If it still stops, restart everything by typing:<br>**docker-compose down**<br>**docker-compose up -d** |

| | | |
|---|---|---|
| **20. Missing .env File (Frontend or Backend)** | The deployment manual asks you to manually create a .env file inside both the Frontend and Backend folders. This file holds important settings like the database password and website address.<br><br>If the file was not created, or if Windows accidentally saved it as ".env.txt" instead of ".env", the system will not find it and will crash. | 1. the Frontend - Deployment folder.<br>2. Look for a file named .env. If you only see .env.txt, that is the problem — Windows added an extra .txt at the end.<br>3. To fix: Right-click the file > Rename > delete the .txt part so only .env remains, then press Enter.<br>4. If the .env file does not exist at all: Right-click inside the folder > Show More Options > New > Text Document.<br>5. Rename it to .env (make sure the .txt extension is fully removed).<br>6. Open the .env file and paste the correct content from the deployment manual. For the frontend, it should include:<br>**VITE_API_BASE_URL= http://localhost:80/api**<br>7. Press Ctrl + S to save, then close it. Repeat the same steps for the Backend - Deployment folder.<br>8. After both .env files are saved, re-run docker-compose up -d --build. |
| **21. Database Table Does Not Exist** | Your MySQL database container is running, but it is completely empty — like a brand new filing cabinet with no folders inside.<br><br>The deployment manual (Step 17) tells you to run a migrate command inside | 1. Make sure all 3 containers are running (green) in Docker Desktop.<br>2. Open Command Prompt as Administrator.<br>3. Enter the backend container by typing: |

| | | |
|---|---|---|
| | the backend container to create the database tables. If that step was skipped, this error will appear when you open the CAPS website. | **docker exec -it caps_backend bash**<br>4. Your Command Prompt prompt will change — this means you are now inside the container.<br>5. Run the migration command to create all the database tables:<br>**php artisan migrate**<br>6. Wait for it to finish. You will see lines like "Migrating... Done" for each table.<br>7. Type exit to leave the container, then refresh the CAPS website in your browser.<br>**exit** |
| **22. Website Loads But Nothing Works (CORS / API Error)** | The frontend (what you see in the browser) and the backend (where data is stored) run on different ports. The browser blocks them from talking to each other unless the backend gives permission.<br><br>This usually happens when the VITE_API_BASE_URL in the frontend .env file does not match the actual address your backend is running on — for example, using the wrong IP address or port number. | 1. Open your Frontend - Deployment folder and open the .env file.<br>2. Check the VITE_API_BASE_URL line. If you are only using CAPS on your own computer, it must be:<br>**VITE_API_BASE_URL= http://localhost:80/api**<br>3. If other devices on the same Wi-Fi need to access it, replace localhost with your IPv4 address. To find it, open Command Prompt and type:<br>**ipconfig**<br>4. Look for IPv4 Address (e.g., 192.168.1.10), then update the .env file to: |

| | | |
|---|---|---|
| | | **VITE_API_BASE_URL= http://192.168.1.10/api**<br>5. Open the Backend - Deployment .env file and make sure SANCTUM_STATEFUL_DOMAINS matches your address:<br>**SANCTUM_STATEFUL_DOMAINS=localhost:80**<br>6. Save both .env files, then rebuild by running:<br>**docker-compose down**<br>**docker-compose up -d --build**<br>7. Open the CAPS website and do a hard reload: press Ctrl + Shift + R. |
| **23. Error 500 Internal Server Error ("The Crash")** | The PHP container failed to resolve database dependencies at runtime. A missing APP_KEY caused the Laravel encryption service to crash entirely. Additionally, incorrect file permissions in the Linux-based Docker environment prevented Laravel from writing to the storage/logs directory, compounding the failure. | 1. Audit Docker logs to identify the failing container and root cause.<br>2. Re-generate the application security key:<br>→ Run: php artisan key:generate<br>3. Synchronize Composer dependencies inside the container:<br>→ Run: composer install (within the PHP container)<br>4. Correct file permissions for the storage/logs directory:<br>→ Run: chmod -R 775 storage bootstrap/cache<br>→ Run: chown -R www-data:www-data storage |

| | | |
|---|---|---|
| | | 5. Restart the Docker containers and verify 500 errors are resolved. |
| **24. Error 204 No Content ("The Silent Success")** | An 'Empty Result Set' logic flaw existed in the original repository. The backend connected to the database successfully but, finding no records (fresh database), returned HTTP 204 'No Content' instead of an empty array — leaving the React frontend Student Dashboard with no data to render. | 1. Identify the API endpoints returning 204 by reviewing controller logic.<br>2. Create a Database Seeding Script to populate baseline records:<br>→ Run: php artisan db:seed<br>3. Update Backend Controllers to always return a standardized JSON structure:<br>→ Return [] (empty array) or {} (empty object) instead of 204.<br>→ Example: return response()->json(['data' => []], 200);<br>4. Ensure the React frontend handles both populated and empty response states gracefully.<br>5. Re-test affected endpoints; confirm 200 OK with valid JSON is returned even when the database is empty. |
| **25. Status 200 Slow Response ("The Snap")** | Although a 200 OK was returned, response times were sluggish because the system performed redundant environment checks on every single request. This inflated the Time to First Byte (TTFB) significantly, making the connection feel broken despite being technically successful. | 1. Profile the request lifecycle to confirm redundant env checks are the bottleneck.<br>2. Apply Environment Variable Sanitization:<br>→ Audit .env file; remove duplicate or unused variables.<br>3. Enable Laravel Configuration Caching: |

| | | |
|---|---|---|
| | | → Run: php artisan config:cache<br>→ Run: php artisan route:cache<br>→ Run: php artisan view:cache<br>4. Verify caching does not interfere with dynamic config values.<br>5. Re-test TTFB to confirm a fast, 'snappy' 200 OK response is achieved. |
| **26. No such file or directory: .env** | Laravel requires an environment file for settings, but it is usually hidden or not included in GitHub for security. | 1. Copy the template file: cp .env.example .env.<br>2. Open .env and update your database name, username, and password to match your Docker settings. |
| **27. The MAC is invalid or No application encryption key** | The system is trying to encrypt data (like passwords) but doesn't have a unique security key yet. | 1. Go into your Laravel folder (or container).<br>2. Run the command: php artisan key:generate.<br>3. Restart your containers to ensure the key is loaded. |
| **28. Class 'App\Models\User' not found / Missing Dependencies** | The project was downloaded from GitHub, but the actual "brains" (libraries) were not included in the download. | 1. Run composer install inside the backend directory.<br>2. Run npm install inside the frontend directory.<br>3. This downloads the thousands of small files needed for Laravel and React to work. |
| **29. Table 'caps_db.users' doesn't exist** | Your database is created, but it is completely empty. There are no tables for the data to go into. | 1. Run the migration command: php artisan migrate.<br>2. This creates the "filing |

| | | |
|---|---|---|
| | | cabinet" structure inside your MySQL container. |
| **30. GET /api/leaderboard returns 500 error** | The backend container is running and routes are registered, but the endpoint aborts before returning data. The likely cause is an incorrect DB host or port for the Tailscale-shared database. | 1. Check backend logs for the specific DB error: docker logs caps_backend_mobile<br>2. Confirm DB_HOST and DB_PORT in .env match the actual Tailscale-shared DB address.<br>3. Test the DB connection from inside the container: docker exec caps_backend_mobile php artisan db:show<br>4. Update .env with the correct values and restart the backend.<br>5. Re-test the endpoint: curl http://localhost:8005/api/leaderboard |